Surviving Result Day: A Measured Decision Guide for Building Examination-Result Websites

Sankalp Pathak and Imran Khan
Department of Computer Science and Engineering, Azad Institute of Engineering and Technology, Lucknow, India
(affiliated to Dr. A. P. J. Abdul Kalam Technical University)
pathaksankalp04@gmail.com, pe.imran@gmail.com

***Abstract*—On result day, lakhs of students try to open an examination board's website at the same moment, and these websites often fail. Boards need evidence to decide how to build them. We built ResultBench, a testbed that runs eleven designs of a result website on the same two processor threads, replays a result-day surge of students who wait and press refresh, and checks every answer. On one machine, a website that builds every page from a database served every student within five seconds at the load it was sized for, but only 30.8 percent at twice that load; splitting it into mirror sites or adding an empty cache did not help. Preparing every result in advance served every student at ten times that load and used 12.5 to 57.8 times less processor time per student, after less than ten seconds of preparation. No design returned a wrong result, and old results after a correction lasted more than a fraction of a second only where cached copies were not updated. Because each student asks for a different result, measures that protect a cache from many identical requests changed nothing, while filling the cache before release served everyone. Impatient students who kept refreshing kept an unprotected website overloaded. A virtual waiting room in front of that website served 98.3 percent of students instead of 43.3 percent, but only after waits of minutes. We turn these measurements into a decision guide for examination boards and release the testbed as open source.**

***Index Terms*—examination results, flash crowds, web architecture, waiting rooms, decision guide**

I. Introduction

When an Indian examination board declares results, lakhs of students try to open the result website at the same time [1]. The release time is announced in advance, almost every request asks for one roll number, and the results do not change while students read them. Few web workloads are simpler, yet result websites keep failing. The chairman of JEE Advanced 2016 described the portal receiving about 10,000 requests per second against a server that could hold 2,000 [2]. On Maharashtra SSC result day in 2021, schools reported that at least half of their students could not see results until evening [1], and the state's education minister apologised [3]. Kenya's examination council reported up to 300,000 hits per second [4], and in 2025 WAEC in Nigeria withdrew its result checker over *wrong* results [5].

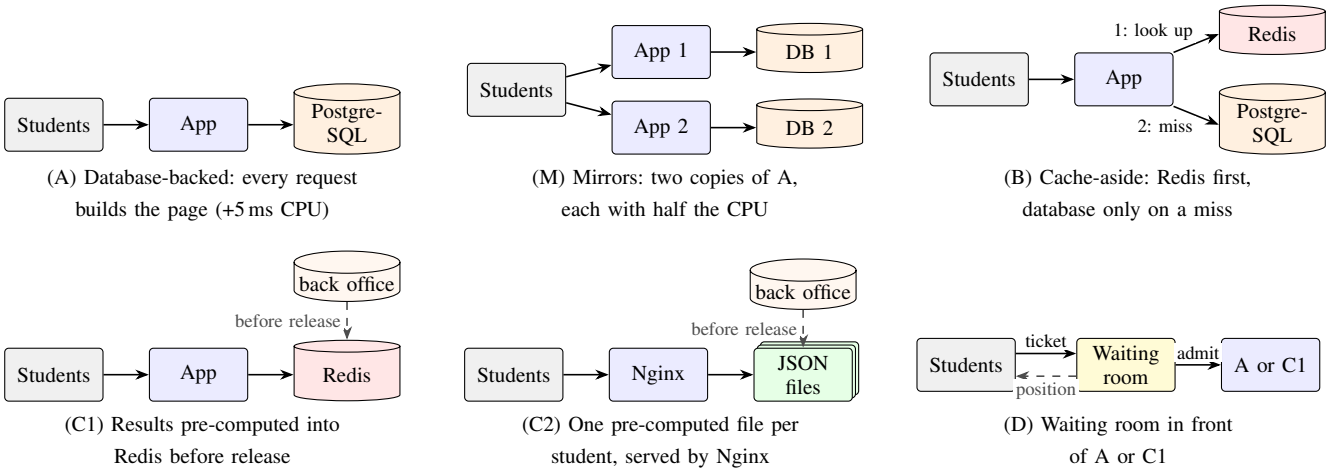
A board that wants to avoid this has to decide how to build its website, and the options are well known: caching or pre-generating pages [6], [7], protecting caches against stampedes [8], [9], admission control [10], [11], and commercial "virtual waiting rooms" that queue visitors during surges [12], [13]. What is missing is evidence about which option works for this workload. Studies of flash crowds look at crowds that are often unforeseen [14], [15], while a result release is announced. The closest recent studies compare a database-only design with a Redis cache for university systems under a fixed number of users, one with a real database [16] and one with simulated delays [17]; neither tests pre-computed results or waiting rooms, models students who refresh, or checks that the answers are correct. We found no peer-reviewed evaluation of waiting rooms at all.

This paper gives boards measured evidence for four decisions: which architecture to use (Q1), whether to put a waiting room in front of it (Q2), how to keep results correct when they are corrected or released early (Q3), and how caches and refreshing students change the load (Q4). We contribute ResultBench, an open-source testbed that runs eleven designs under one resource budget, replays a result-day surge of students who refresh, and checks every answer against ground truth; a controlled comparison of the designs; and a decision guide that turns the measurements into advice (Table II).

II. Methods

A. Designs

Every design answers `GET /result?roll=`n with the same JSON body and gets the same budget of two CPUs (hardware threads) for answering students (Fig. 1). Before the announced release time every design answers "not released". **A** builds each page from PostgreSQL (a 4-table join). **M** splits the budget into two independent copies of A ("mirror links"); each student picks one at random and, on retry, switches with probability 0.5. **B0** puts a Redis cache in front of A (cache-aside); **B1** adds request coalescing, **B2** probabilistic early recomputation [8], and **B3** fills the cache before release. **C1** pre-computes every result into Redis, and **C2** writes one JSON file per student, served by Nginx; both keep an idle back-office PostgreSQL (0.25 CPU, counted in cost). **D** puts a waiting room (0.25 CPU) in front of A, C1 or C2 (we test D-A and D-C1). It admits students at 80% of the backend's capacity, either first come, first served (FIFO) or by a lottery among early arrivals, as some products offer [18]. Its ticket is either *naive* (held in the page and lost when the student presses refresh) or a signed *token* cookie (kept on refresh). An admitted student may use the backend for 60 s; the waiting room limits neither how many admitted students are active nor how many requests each sends.



(A) Database-backed: every request builds the page (+5 ms CPU)

(M) Mirrors: two copies of A, each with half the CPU

(B) Cache-aside: Redis first, database only on a miss

(C1) Results pre-computed into Redis before release

(C2) One pre-computed file per student, served by Nginx

(D) Waiting room in front of A or C1

Fig. 1. How each design answers a request. Solid arrows are on the request path; dashed arrows happen before release (C1, C2) or tell a queued student their position (D).

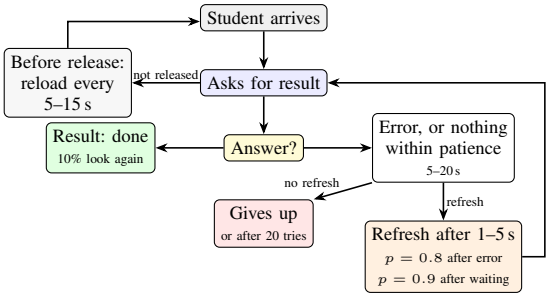


Fig. 2. How each simulated student behaves. All numbers are assumptions; Section III-D varies them.

All application designs run the same Go server, which works on at most 256 requests at once and lets 4096 more wait, as real servers do [19]. Real portals run full web frameworks; to stand in for that work, building one page from the database costs 5 ms of CPU. This value is an assumption, which we vary in Section III-A.

B. Students and surge

The data are 200,000 synthetic students with six subjects each, generated from a seed so that the right answer for any roll number can be recomputed; no real student data was used. Students arrive as an open-loop Poisson process [20]: the rate rises over the 20 s before release to 20% of the peak, reaches the peak within 3 s of release and decays with a time constant of 30 s over 90 s. Real result days stay busy far longer [1]; we compress the surge to keep runs short. Early students reload every 5–15 s until release. Each student waits 5–20 s for an answer; after an error they refresh with probability 0.8, after waiting in vain with probability 0.9, after a pause of 1–5 s, for at most 20 attempts. These numbers are assumptions; Section III-D varies them.

C. Testbed and measurement

All runs used one desktop computer (AMD Ryzen 7 5800X, 8 cores, 32 GB, Windows 11 with WSL 2 and Docker 29.9). The design under test was pinned to the two hardware threads of one core and the load generator to twelve threads on six other cores. We first found design A's capacity: the highest constant arrival rate at which 99% of students got a correct result within 2 s, 223 students/s. Loads are multiples of it; the JEE 2016 incident was about 5× [2]. An oracle classifies every response as correct, stale (the old version after a correction), wrong, missing ("not found" for a real student) or failed. We report the share of students who got a correct result within 5 or 10 s of their first request after release, or at all, and the CPU-seconds used by all containers per 1,000 students served. Each configuration ran 3 times in random order; we report medians with 95% bootstrap intervals. Against k6's constant-arrival-rate executor [21] at 150 requests/s, our generator measured median latencies of 5.3–5.5 ms against 5.4 ms. Six waiting-room runs in which the generator ran out of local ports were discarded and run again with a wider port range.

III. Results and Discussion

No run in any experiment returned a wrong result, and no container crashed.

A. Q1: Which architecture?

Fig. 3 compares five designs from 1× to 10× A's capacity, with students who do not refresh so that every design faces the same requests. A served every student within 5 s at 1× but only 30.8% at 2×, 7.7% at 5× and 1.4% at 10×; at 10× only 20.6% got a result at all. Mirrors (M) matched A at 2× and did worse beyond it, using up to 1.64 times A's CPU per student. A cold cache (B0) was worse than no cache, serving only 70.8% within 5 s even at 1×. The pre-computed designs C1 and C2 served every student at every load, with 99th-percentile latencies below 0.7 ms, and used 12.5 to 57.8 times less CPU per student than A. C1's capacity is at least 7,500 students/s (33.6 times A's), where our generator ran out. Pre-computing all 200,000 results took 6.9–7.0 s (C1) and 9.4–9.8 s (C2). With a page-building cost of 2 ms or 0 ms instead

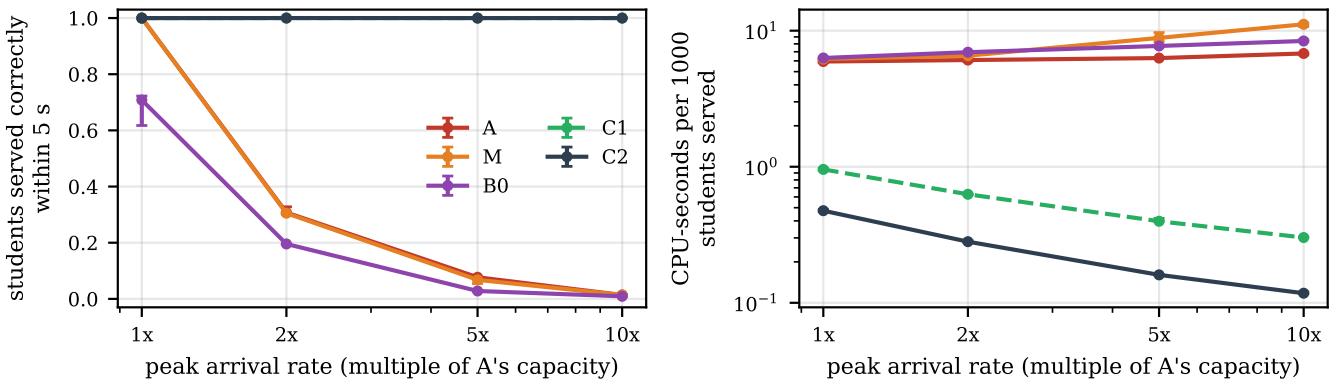


Fig. 3. Q1. Left: share of students who got their correct result within 5 s. Right: CPU-seconds per 1,000 students served (log scale). Medians of 3 runs with 95% bootstrap intervals; students do not refresh in these runs. C1 (dashed) and C2 overlap at 100%.

TABLE I
DESIGN A ALONE AND BEHIND A WAITING ROOM AT 5× WITH REFRESHING STUDENTS (MEDIAN OF 3 RUNS; RUN ENDS 135 S AFTER RELEASE).

Waiting room	Served (%)		Waiting	Wait	τ
	10 s	at all	at end (%)	p50 (s)	
none (A alone)	6.1	43.3	0.0	0.2	0.09
FIFO, token	4.1	36.8	52.2	40.6	0.79
FIFO, naive	3.7	41.9	58.1	59.5	0.60
lottery, token	4.4	34.2	52.5	38.2	0.01
lottery, naive	3.8	41.8	58.1	59.8	-0.01

of 5 ms, A and B0 served at least 99.8% of students within 5 s at the same load as 2×, so this assumption decides how much load a database-backed portal can take. The cost order did not change: per student served, A used 2.6 (0 ms) to 9.7 (2 ms) times the CPU of C2, and B0 always used more than A.

B. Q2: Does a waiting room help?

Table I shows A alone and behind each waiting room at 5× with refreshing students. Within this 135 s run the waiting room served no more students than A alone (34.2–41.9% against 43.3%), but A's unserved students had given up while 52–58% of the waiting room's were still queued. We therefore ran FIFO again with 240 s more for the queue to empty. It then served 98.3% of students with naive tickets, but the median student waited 121 s to be admitted (95th percentile 282 s); in our model, queued students wait up to 10 minutes, and real ones may leave sooner. With tokens it served only 74.5%: admitted students who ran out of patience refreshed straight at A, which received 3.21 requests per student instead of 1.07. Only FIFO with tokens kept arrival order (Kendall's $\tau = 0.79$ between arrival and admission order for early students, Table I); naive tickets lowered it to 0.60 because a refresh sent the student to the back, and the lottery gave $\tau \approx 0$, like A alone. In front of C1, which needs no protection, the waiting room only added delay and raised the CPU per 1,000 students from 0.44 to 0.77–0.78 CPU-seconds.

C. Q3: Are the results correct?

We published corrections for 1% of students 30 s after release at 1×. Most students look once and had already looked, so corrected students received only about 90 answers after the correction in the three runs of each design. A, B0 with cache invalidation, and C1 gave at most one old result, while the correction was being written (it finished within 0.19 s). B0 without invalidation served old results for up to 10.89 s, and would until its 10-minute lifetime ended; C2 served old files from Nginx's 30-second file cache for up to 13.69 s. When only 90% of results were ready at release, C1 wrote the rest in 0.08 s and C2 in 1.63 s, and one student was told "not found"; a slower preparation would leave more students without a result for longer.

D. Q4: Caches and refreshing students

At 2×, with every student asking for a different roll number, every cache miss in B0 was the first request for that key; no two requests ever raced for the same key. Request coalescing (B1) and early recomputation (B2) therefore changed nothing (8.5–8.9% served within 5 s), while filling the cache before release (B3) served everyone, at 0.71 CPU-seconds per 1,000 students against B0's 6.27.

Refreshing changes the load a lot (Fig. 4). For A at 2×, students who never refresh sent 1.17 requests each; our default students sent 2.23, so more were served in the end (88.1% against 77.7%) but fewer within 10 s (32.7% against 53.7%). Impatient students (patience 3–8 s) sent 8.65 requests each and kept A overloaded until the end of every run; only 22.1% were served. Behind a waiting room the load on A stayed near one request per student with default students (1.32), but with impatient students A still received 5.30 requests per student.

E. Decision guide

Table II turns these results into advice. The numbers come from one machine and should be read as relative, not absolute.

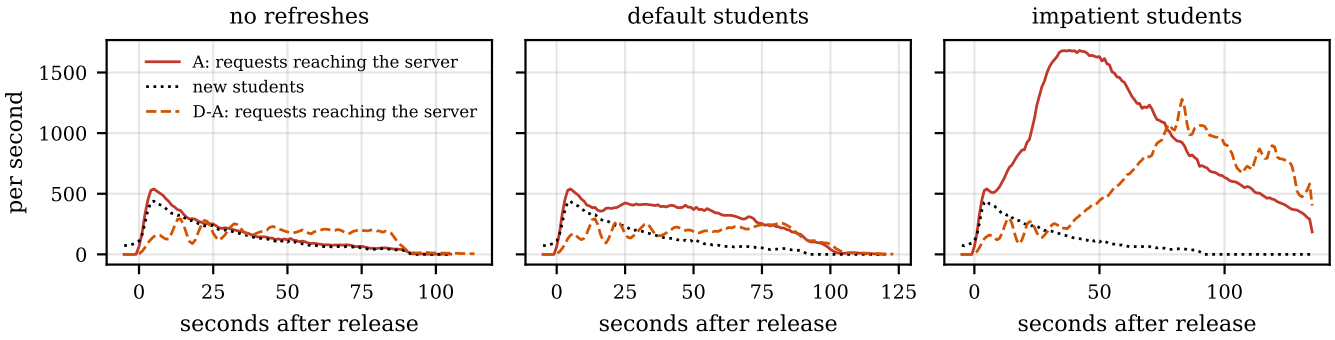


Fig. 4. Q4. Requests reaching the server each second for A alone (solid) and behind a waiting room (D-A, dashed) at 2×, against new students (dotted). One run per kind of student.

TABLE II
DECISION GUIDE FOR EXAMINATION BOARDS, FROM OUR MEASUREMENTS

If the board…	then…	because (section)
has the results before release	publishes them pre-computed, as files or in a key–value store	served 10× at 12.5–57.8× less CPU (III-A)
must build pages from a database	does not count on mirrors or an empty cache; fills the cache before release	M and B0 no better than A; B3 served all (III-A, III-D)
cannot finish preparing by release time	delays the release	early students are told “not found” (III-C)
corrects results after release	deletes or rewrites every cached copy and keeps file caches short	old results for up to 14 s otherwise (III-C)
expects students to refresh	makes the page retry by itself, with back-off	impatient students kept A down (III-D)
considers a waiting room in front of a slow website	uses one only if students can wait minutes; admits FIFO; does not let admitted students refresh without limit	served 98.3% instead of 43.3%, after a 121 s median wait; 74.5% with tokens (III-B)

Limitations. All runs used one machine, no content delivery network and a surge compressed to 90 s. The students and the page-building cost are models. Three runs per configuration give wide intervals for some measures. We have not yet measured a real result day.

IV. Conclusion

With ResultBench we measured eleven designs of a result website under the same budget. Building pages on request failed by twice the capacity of a database-backed portal, while pre-computed results served ten times that load at a small fraction of the CPU. Wrong results never appeared, and old results lasted more than a moment only where cached copies were not updated. Stampede defences do not help when every student asks for a different result, and refreshing students can keep a portal overloaded. A waiting room in front of an overloaded portal served almost everyone, but only after waits of minutes, and fewer when admitted students could keep refreshing. Next, we plan to measure real result days and to test real portal software and multi-server deployments. ResultBench is open source (MIT licence) at https://github.com/K144U/NoCrashing.

Acknowledgments

An AI assistant (Claude Opus 5.5, Anthropic) was used under the authors’ direction for software development, data analysis, figures, and drafting and editing of the text. The authors reviewed all content and are responsible for it.

References

[1] “Server crash hassles scores of SSC students,” Hindustan Times (Mumbai), 17 July 2021, https://www.pressreader.com/india/hindustan-times-st-mumbai/20210717/281513639165468, 2021.

[2] M. Mehta, “JEE advanced portal to be functional soon, says chairman of JEE advanced,” PaGaLGuY, 12 June 2016, https://www.pagalguy.com/articles/jee-advanced-portal-to-be-functional-soon-says-chairman-of-jee-advanced-2679, 2016.

[3] S. Yechuri, “Varsha Gaikwad apologizes for Maharashtra results website crash,” Careerindia, 16 July 2021, https://www.careerindia.com/news/varsha-gaikwad-apologizes-for-maharashtra-results-website-crash-029945.html, 2021.

[4] Kenya National Examinations Council, “Statement on the 2024 KCSE results portal,” Post on X, 9 January 2025, https://x.com/KNECKenya/status/1877322346736017641, 2025.

[5] M. Usman, “WAEC shuts down results portal, reviews 2025 WASSCE results,” International Centre for Investigative Reporting, 7 August 2025, https://www.icirnigeria.org/waec-shuts-down-results-portal-reviews-2025-wassce-results/, 2025.

[6] J. Challenger, A. Iyengar, and P. Dantzig, “A scalable system for consistently caching dynamic web data,” in *Proc. IEEE INFOCOM*, 1999, pp. 294–303.

[7] A. Labrinidis and N. Roussopoulos, “WebView materialization,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2000, pp. 367–378.

[8] A. Vattani, F. Chierichetti, and K. Lowenstein, “Optimal probabilistic cache stampede prevention,” *Proc. VLDB Endowment*, vol. 8, no. 8, pp. 886–897, 2015.

[9] R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H. C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab, D. Stafford, T. Tung, and V. Venkataramani, “Scaling memcache at Facebook,” in *Proc. 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013, pp. 385–398.

[10] M. Welsh, D. Culler, and E. Brewer, “SEDA: An architecture for well-conditioned, scalable internet services,” in *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 230–243.

[11] S. Elnikety, E. Nahum, J. Tracey, and W. Zwaenepoel, “A method for transparent admission control and request scheduling in e-commerce web sites,” in *Proc. 13th International World Wide Web Conference (WWW)*, 2004, pp. 276–286.

[12] Cloudflare, “Cloudflare waiting room,” Developer documentation, https://developers.cloudflare.com/waiting-room/, accessed October 2026.

[13] J. Pirtle, J. Morgan, and J. Thario, “Introducing AWS virtual waiting room,” AWS Compute Blog, 10 February 2022, https://aws.amazon.com/blogs/compute/introducing-aws-virtual-waiting-room/, 2022.

[14] J. Jung, B. Krishnamurthy, and M. Rabinovich, “Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites,” in *Proc. 11th International World Wide Web Conference (WWW)*, 2002, pp. 293–304.

[15] P. Wendell and M. J. Freedman, “Going viral: Flash crowds in an open CDN,” in *Proc. ACM Internet Measurement Conference (IMC)*, 2011, pp. 549–558.

[16] M. A. S. Putranto and A. Pinandito, “Analisis kinerja cache Redis pada RESTful API berbasis Express.js,” *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer (J-PTIIK)*, vol. 10, no. 2, 2026.

[17] Y. A. Tofan, R. Parlika, and S. P. Krispriyanto, “Perancangan arsitektur cache-aside menggunakan Redis pada sistem informasi akademik: Studi prototipe dan simulasi beban,” *Jurnal Informatika Polinema (JIP)*, vol. 12, no. 3, pp. 493–502, 2026.

[18] Cloudflare, “Queueing method,” Cloudflare Waiting Room documentation, https://developers.cloudflare.com/waiting-room/reference/queueing-methods/, accessed October 2026.

[19] The Apache Software Foundation, “Apache MPM common directives: MaxRequestWorkers,” Apache HTTP Server 2.4 documentation, https://httpd.apache.org/docs/2.4/mod/mpm_common.html, accessed October 2026.

[20] B. Schroeder, A. Wierman, and M. Harchol-Balter, “Open versus closed: A cautionary tale,” in *Proc. 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006.

[21] “Open and closed models,” Grafana k6 documentation, https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/, accessed October 2026.